TripSync

Wersja 1.1: plan, budżet, mapa i pogoda - także dane offline.



Cel i zakres

Aplikacja dla niewielkich grup znajomych organizujących wspólne wyjazdy. Łączy harmonogram, zadania, pakowanie, pomysły, budżety, kategorie wydatków, spłaty, historię zmian oraz mapę i prognozę pogody. Dane są dostępne bez internetu, a po odzyskaniu połączenia synchronizowane z własnym backendem REST.

Zakres dla trzech osób rozszerza podstawowe wymagania o współpracę wielu kont, role i kontrolę dostępu, wykrywanie konfliktów edycji, kolejkę operacji, dokładne rozliczenia i integracje Open-Meteo/OpenStreetMap.

Platforma	Dostarczany rezultat
Android 8.0+ / Kotlin	Kod źródłowy, APK debug, backend SQLite/Python
MVVM / Jetpack Compose / Material 3	Ekrany, testy, diagramy, instrukcja i screeny
Działanie lokalne i online	Demo bez konta; współdzielenie po uruchomieniu API

Scenariusz użytkownika

Organizator tworzy wyjazd i udostępnia kod zaproszenia. Grupa ustala plan, głosuje na pomysły i dzieli zadania. Każdy zapisuje poniesione koszty, także offline. Po synchronizacji wszyscy widzą aktualne dane, a ekran kosztów pokazuje salda i propozycje przelewów.

Granice projektu

Jedna waluta na wyjazd: PLN, EUR albo USD. Bez czatu, rezerwacji, realizowania płatności i pełnych map offline. Synchronizacja jest okresowa lub ręczna; aplikacja nie deklaruje aktualizacji w czasie rzeczywistym.

Wymagania

Funkcjonalne

ID	Wymaganie
F01	Konta: rejestracja, logowanie, szyfrowana sesja i wylogowanie.
F02	CRUD wyjazdów: nazwa, miejsce, daty, opis i waluta; rola organizatora.
F03	Uczestnicy: dołączenie kodem/linkiem, rotacja kodu, usuwanie dostępu.
F04	Harmonogram: CRUD, godzina, miejsce, kolejność i wykonanie.
F05	Propozycje: CRUD, jeden własny głos, cofnięcie i ponowne oddanie głosu.
F06	Przeniesienie propozycji do harmonogramu przez formularz.
F07	Zadania i pakowanie: CRUD, przypisanie osoby, oznaczanie wykonania.
F08	Wydatki: CRUD, płatnik, data, uczestnicy i dwa sposoby podziału kosztu.
F09	Salda i propozycje przelewów z dokładnością grosza/centa.
F10	Trwałość danych i lokalny CRUD bez internetu.
F11	Synchronizacja, kolejka operacji, ponowienia i konflikty wersji.
F12	Lokalne przypomnienia o punktach planu i zadaniach.
F13	Wieloekranowa nawigacja, zachowanie formularza, ustawienia motywu.
F14	Walidacja i czytelne komunikaty błędów sieci, bazy oraz serwera.
F15	Edytowalne demo bez konta i systemowe udostępnianie planu.

Niefunkcjonalne

N01-N03: Material Design 3, responsywne widoki, modularny kod i asynchroniczne operacje Room/Retrofit poza głównym wątkiem. **N04:** odporność na odtworzenie aktywności przez ViewModel, rememberSaveable i Room. **N05-N06:** kontrola dostępu, Keystore, HTTPS, dostępność i etykiety kontrolek. **N07-N09:** testy domeny, UI i integracji, możliwość rozbudowy, historia Git i komplet materiałów do uruchomienia.

Rozszerzenia wersji 1.1

ID	Wymaganie
F16	Budżet całego wyjazdu i limity kategorii; CRUD przez organizatora.
F17	Sześć kategorii wydatków, zestawienie i filtr kosztów.
F18	CRUD własnych deklaracji spłat oraz przeliczenie pozostałych sald.
F19	Historia zmian z autorem i czasem serwera; wyłącznie do odczytu.
F20	CRUD współdzielonych miejsc, mapa i otwarcie aplikacji map.
F21	Wyszukiwanie miejscowości przez API i zapis na wspólnej mapie.
F22	Prognoza na 7 dni: temperatury, opis pogody i szansa opadów.
F23	Ostatnia prognoza zapisana lokalnie, daty i komunikaty błędów.

Budżety i spłaty

Organizator ustala limit całkowity oraz limity sześciu kategorii. Suma limitów kategorii nie może przekraczać budżetu. Ekran pokazuje wykorzystanie i przekroczenia. Zapisane spłaty zwiększają saldo nadawcy i zmniejszają saldo odbiorcy, dzięki czemu proponowane rozliczenie uwzględnia oddane pieniądze. Nadpłata tworzy należność w przeciwnym kierunku.

Historia i miejsca

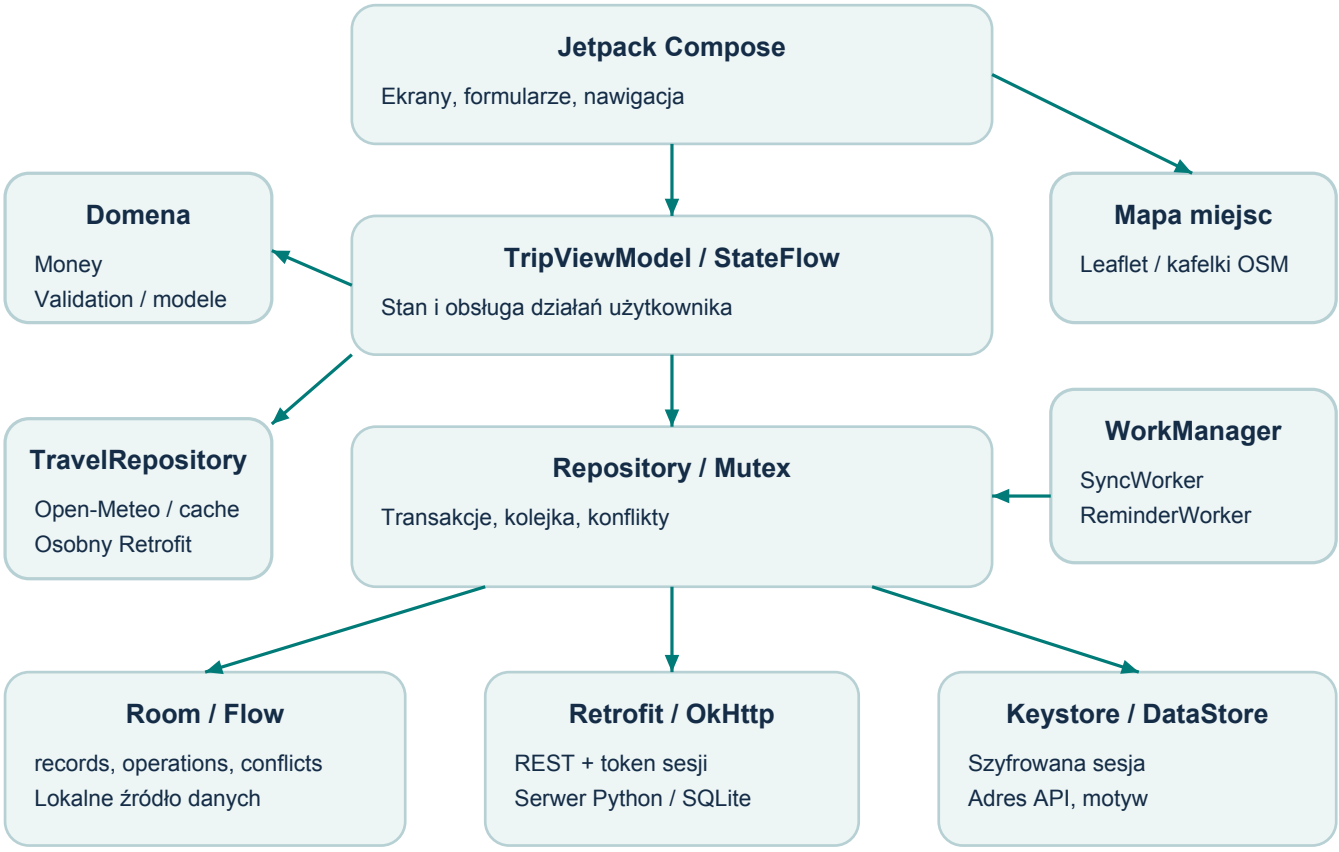
Historia powstaje na serwerze w transakcji zmiany; ponowienia i konflikty nie tworzą duplikatów. Klient nie może wysyłać wpisów historii. Pokazujemy 200 najnowszych wpisów, bez cofania zmian. Nazwy zapisanych miejsc można wybrać w harmonogramie i pomysłach.

Pogoda i granice integracji

Open-Meteo zwraca prognozę najbliższych siedmiu dni w strefie miejsca. Wyszukiwanie obejmuje miejscowości; dokładne punkty można dodać współrzędnymi. Mapa wymaga internetu do kafelków, a stara prognoza jest oznaczona datą i komunikatem. Spłaty są deklaracjami użytkowników, bez wykonywania lub bankowego potwierdzania przelewów.

Architektura aplikacji

MVVM z wydzieloną domeną



UI odczytuje Room. Sieć aktualizuje lokalną bazę; nie blokuje ekranu.

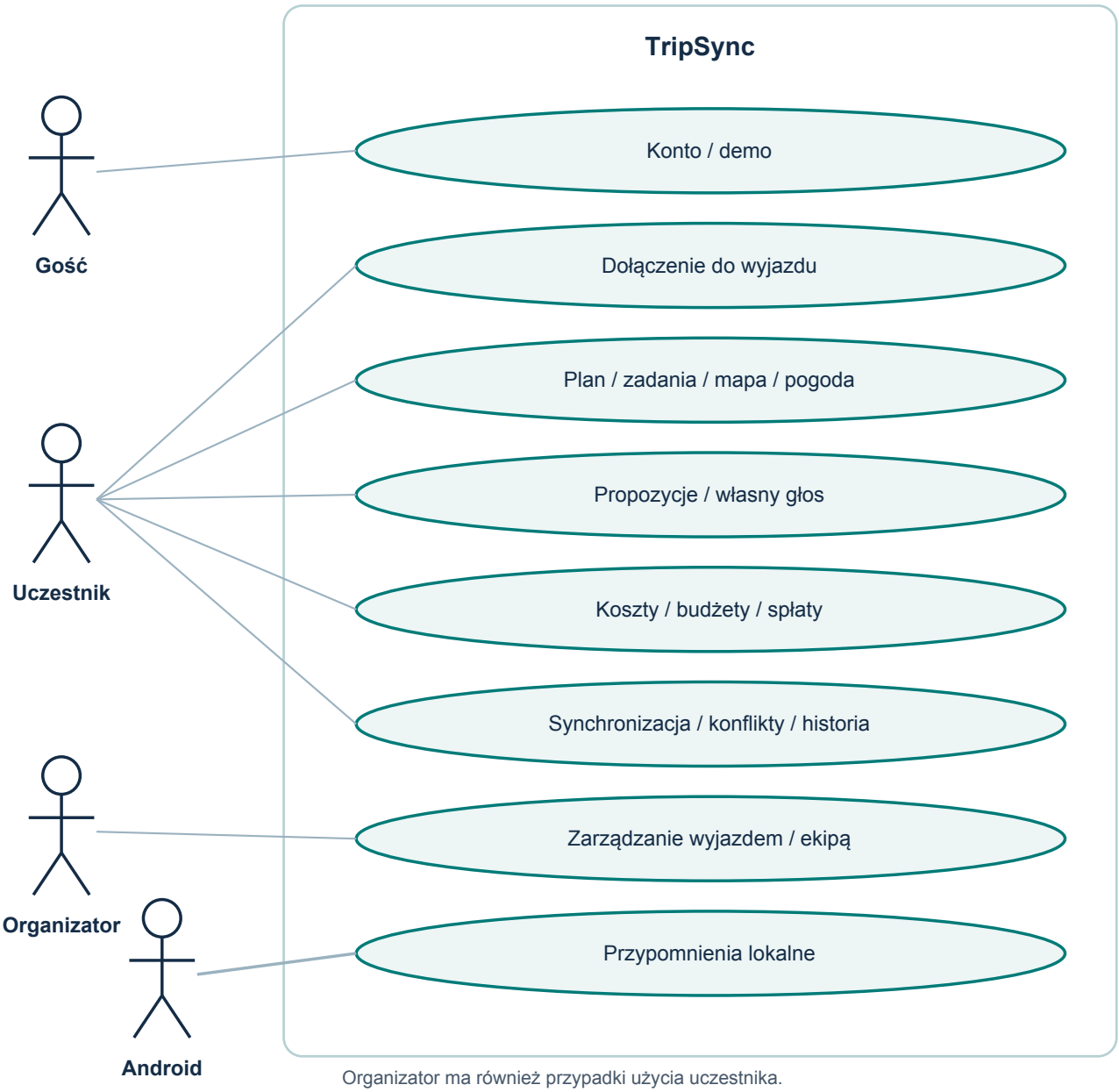
Przepływ i synchronizacja

1. Formularz waliduje dane, a repozytorium zapisuje rekord i operację w jednej transakcji Room.
2. Ekrany otrzymują dane z lokalnej bazy przez Flow i StateFlow.
3. SyncWorker wysyła operacje, a serwer sprawdza sesję, rolę, dane i bazową wersję.
4. Identyfikator operacji zapobiega podwójnemu zapisowi po ponowieniu.
5. Różnica wersji tworzy konflikt. Użytkownik wybiera lokalną albo zdalną wersję.
6. Snapshot uzupełnia Room, zachowując niewysłane lokalne zmiany. Utrata dostępu usuwa lokalny wyjazd po udanej synchronizacji.

Mapa: WebView + lokalny Leaflet + kafelki OSM. Pogoda: TripViewModel → TravelRepository → osobny Retrofit Open-Meteo; prywatny cache prognozy. Nowe typy rekordów nie zmieniają schematu Room.

Usunięcia są przesyłane jako tombstone. Mutex porządkuje zapisy i synchronizację. Dane z aktywnym konfliktem czekają na rozstrzygnięcie; nie są automatycznie nadpisywane.

Przypadki użycia



Organizator ma również przypadki użycia uczestnika.

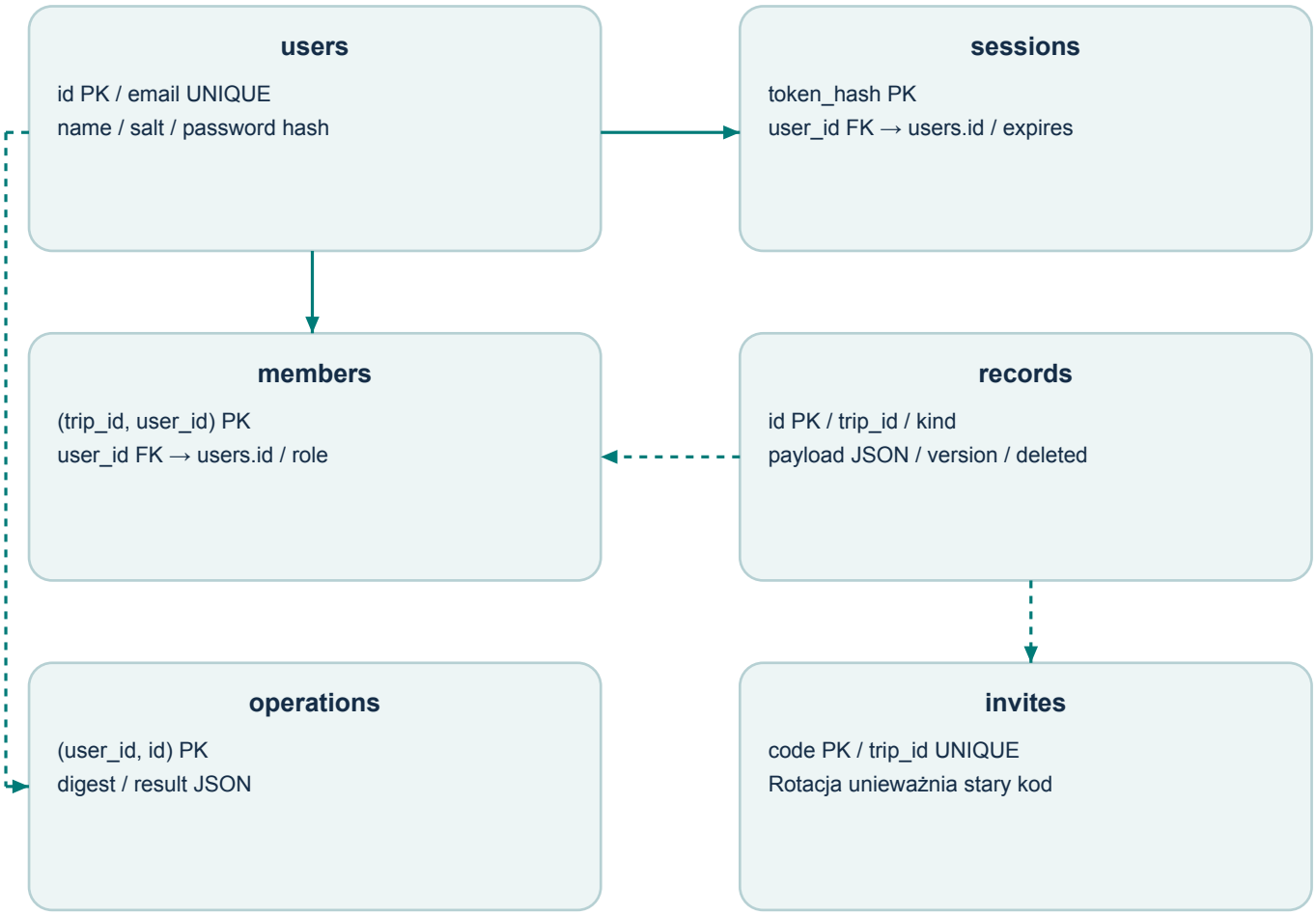
Role i uprawnienia

Gość: konto albo lokalne demo. **Uczestnik:** wspólny plan, zadania, koszty i propozycje; głos wyłącznie za siebie. **Organizator:** dodatkowo tworzenie, edycja i usuwanie wyjazdu oraz zaproszenia, zarządzanie uczestnikami i budżetem.

System Android: WorkManager uruchamia synchronizację i lokalne przypomnienia. Organizator nie może usunąć siebie ani uczestnika mającego aktywne wydatki; wcześniej należy zmienić koszty i usunąć aktywne spłaty tej osoby.

Diagramy baz danych

Backend: fizyczne tabele SQLite



Strzałki ciągłe: fizyczne powiązania FK do users. Linie przerywane: powiązania logiczne obsługiwane w API. Rekordy podrzędne wskazują wyjazd przez trip_id; payload JSON zawiera pola danego typu.

Urządzenie: Room



Relacje lokalne są logiczne, bez fizycznych FK. Unikalny indeks operations.recordId zapewnia jedną oczekującą operację na rekord. Indeksy records.tripId i records.kind wspierają odczyty. Spójność zapisów zapewniają transakcje repozytorium. Schemat Room jest dołączony w app/schemas.

Technologie i uruchomienie

Technologia	Zastosowanie
Kotlin / AGP 9.1.1 / SDK 36.1	Android; Gradle Wrapper 9.3.1, JDK 17 lub 21
Compose BOM 2024.01.00 / Material 3	Interfejs, motyw, kontrolki i dostępność
Navigation Compose 2.7.7 / Lifecycle 2.9.0	Nawigacja i ViewModel; zależności rozwiązywane przez Gradle
Room 2.8.4 / KSP 2.2.10-2.0.2	Lokalna baza i generowany DAO
Coroutines 1.10.2 / Flow	Asynchroniczny odczyt i reaktywny stan
Retrofit 2.9.0 / OkHttp 4.12.0 / Gson 2.11.0	REST oraz kodowanie JSON
DataStore 1.1.7 / WorkManager 2.9.0	Ustawienia, synchronizacja i przypomnienia
Keystore / AES-GCM	Szyfrowane przechowywanie sesji
Python 3.10+ / SQLite / unittest	Backend bez dodatkowych pakietów
JUnit 4 / Compose UI Test / AndroidJUnitRunner	Testy jednostkowe i urządzeniowe
Leaflet 1.9.4 / OpenStreetMap	Lokalna biblioteka JS i kafelki mapy
Open-Meteo / TravelRepository	Geokodowanie, prognoza i cache bez klucza API

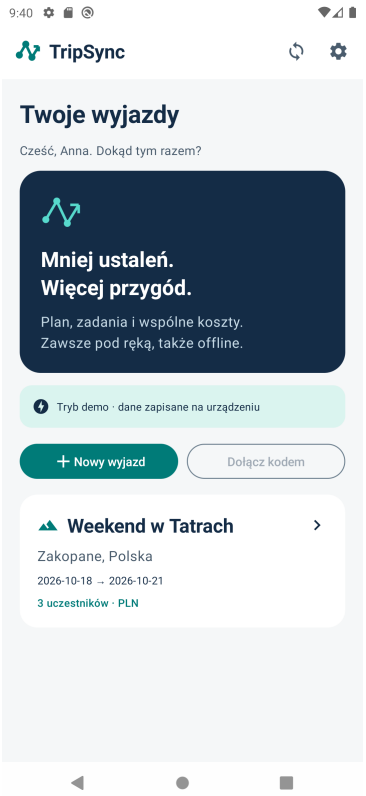
Instalacja i start

Najprościej: zainstaluj **output/apk/TripSync-debug.apk** i wybierz **Wypróbuj bez konta**. Do współpracy uruchom serwer, ustaw adres API przed zalogowaniem i utwórz oddzielne konta.

```
python3 backend/server.py
./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
```

Emulator debug: http://10.0.2.2:8080/. Telefon przez USB: adb reverse tcp:8080 tcp:8080 i adres http://127.0.0.1:8080/. Pozostałe urządzenia wymagają API HTTPS z zaufanym certyfikatem. Otwórz katalog w Android Studio i ustaw własne sdk.dir w local.properties. Pełne komendy, uruchomienie TLS i testy są w README.md.

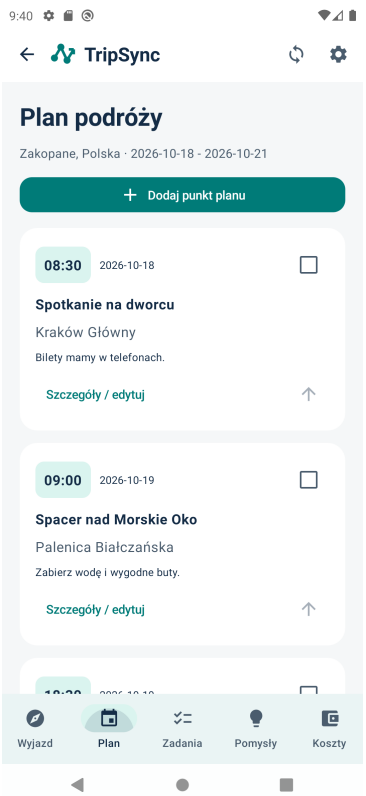
Najważniejsze widoki / 1



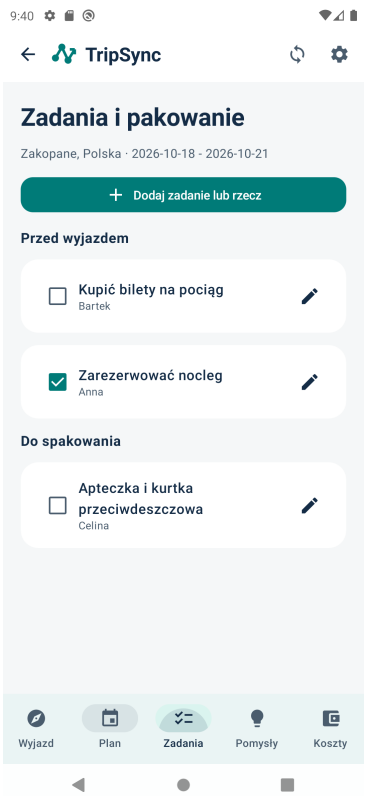
Lista wyjazdów i status danych



Podsumowanie wyjazdu



Harmonogram i wykonanie

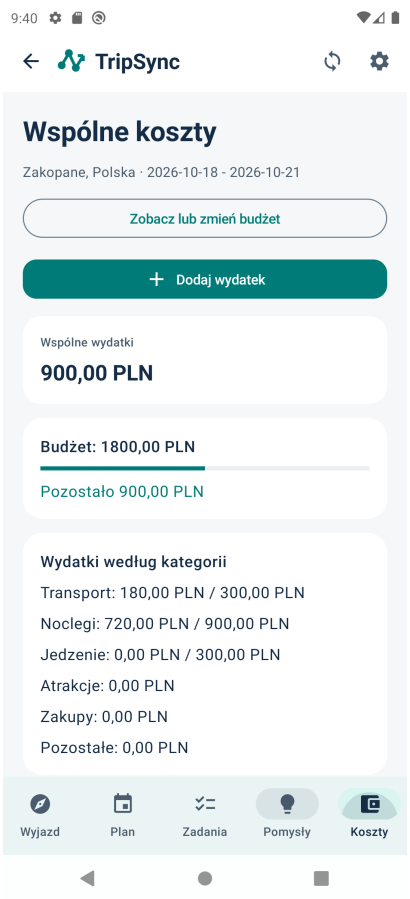


Zadania oraz pakowanie

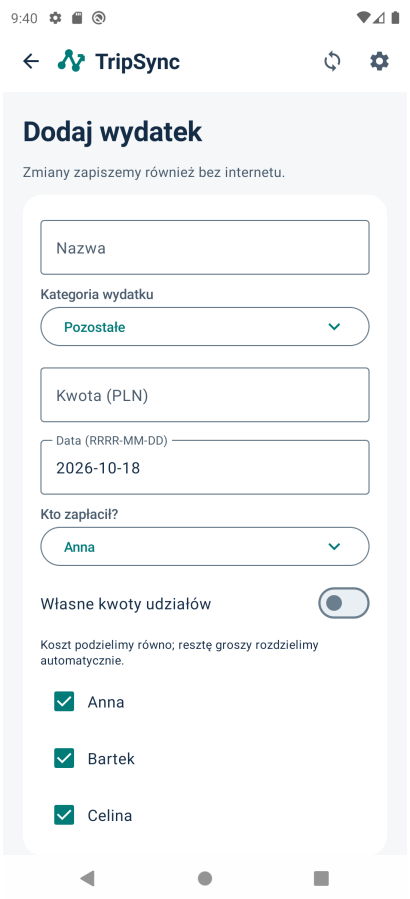
Najważniejsze widoki / 2



Propozycje i głosowanie



Wydatki, budżet i kategorie



Formularz wydatku

Źródło zrzutów

Obrazy pochodzą z działającej aplikacji na emulatorze Android, a nie z makiet. Test UI przechodzi przez główne zakładki i zapisuje rzeczywiste screeny. Dane przykładowe: trzy osoby, nocleg 720 PLN i bilety 180 PLN.

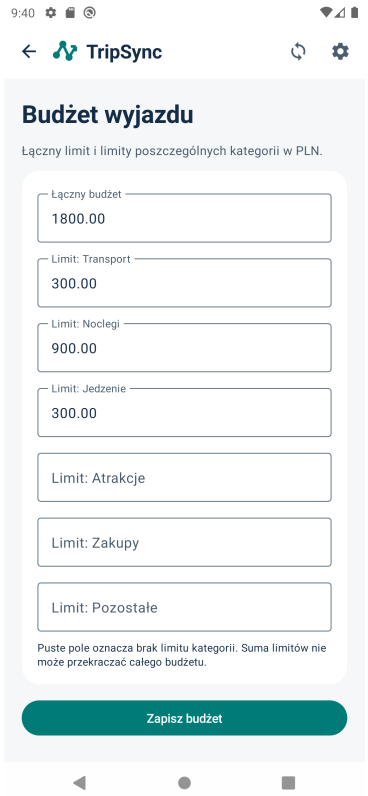
Interpretacja rozliczeń

Przy równym podziale kosztów Anna ma saldo +420 PLN, Bartek -120 PLN, a Celina -300 PLN. Propozycja rozliczenia: Bartek oddaje Annie 120 PLN, Celina oddaje Annie 300 PLN. Suma sald wynosi zero.

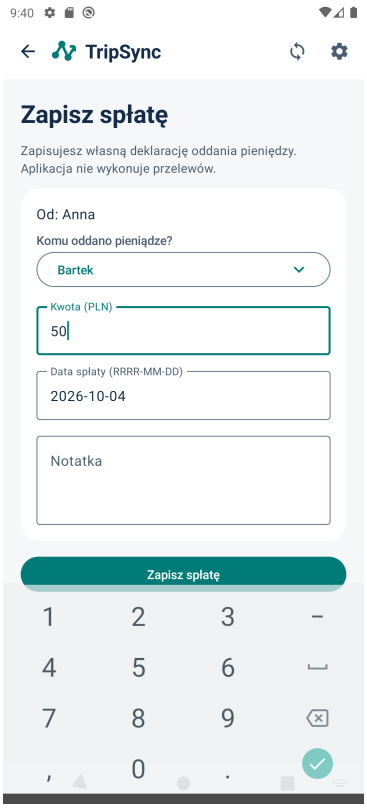
Nawigacja

Poza pokazanymi widokami dostępne są logowanie/rejestracja, uczestnicy z zaproszeniami, formularze pozostałych danych, ustawienia oraz rozstrzyganie konfliktów.

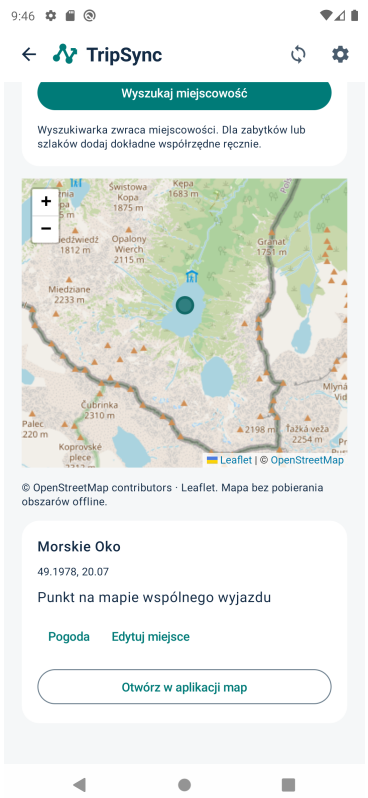
Najważniejsze widoki / 3



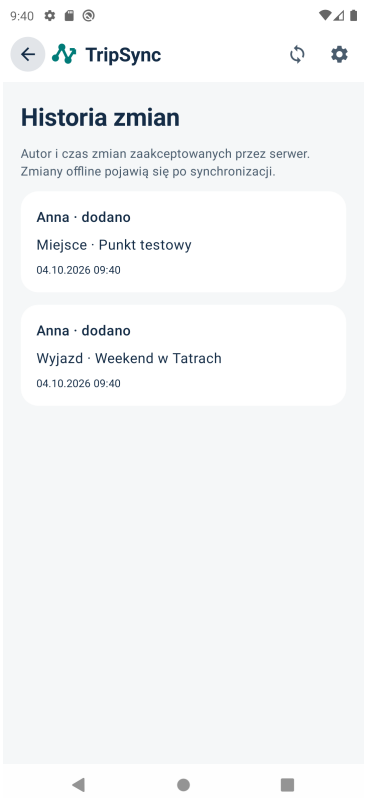
Limity całego budżetu i kategorii



Formularz własnej spłaty

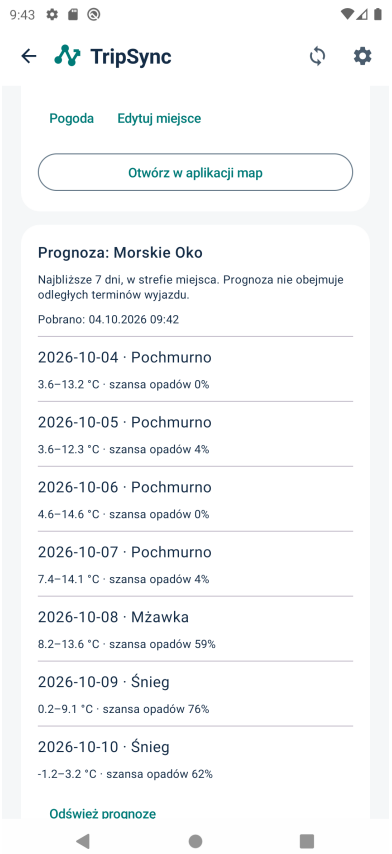


Mapa wspólnych miejsc



Autorzy i czas zmian

Pogoda i darmowy hosting



Rzeczywista prognoza Open-Meteo

PythonAnywhere Beginner Free

Backend ma gotowy adapter WSGI. Trzy pliki Python, trwała SQLite poza kodem, bez dodatkowych pakietów. HTTPS i domenę aplikacji zapewnia hosting.

1. Załóż darmowe konto.
2. Wgraj server.py, wsgi.py i backup.py do ~/TripSync/backend.
3. Web → Manual configuration → Python 3.10+.
4. Wklej szablon deploy/pythonanywhere_wsgi.py.example do WSGI.
5. Reload i sprawdź /health.
6. Wpisz adres HTTPS w TripSync przed rejestracją.

Baza: ~/tripsync-data.sqlite. Aktualizacja: kopia bazy, nowe pliki kodu, Reload. Pilnuj terminu odnowienia aplikacji w panelu. Pełne komendy i odtwarzanie kopii: docs/hosting.md.

Hosting nie został publicznie uruchomiony. Adapter WSGI przetestowano lokalnie. Konto i adres należą do użytkownika.

Źródła i licencje

PythonAnywhere: darmowy plan, jedna aplikacja i 512 MB plików · zmiany planu od 2026 r.. Darmowy Render nie zachowuje lokalnej SQLite po restartach; wybór hostingu uwzględnia trwałość danych.

Open-Meteo: bezpłatne API niekomercyjne, dane CC BY 4.0. OpenStreetMap: polityka kafelków i atrybucja. Leaflet: licencja BSD-2-Clause w assets. Mapa nie pobiera całych obszarów offline i nie wymaga lokalizacji GPS telefonu.

Testy i rezultaty

Weryfikacja wykonana 04.10.2026

Obszar	Wynik
Backend Python	22 testów / 0 niepowodzeń
Domena Kotlin / JUnit	18 testów / 0 niepowodzeń
Android: interfejs i REST	16 testów / 0 niepowodzeń
Budowanie APK	assembleDebug: zakończone powodzeniem
Android Lint	0 błędów / 1 ostrzeżenie

Zakres sprawdzeń

Domena: dokładne kwoty, reszty groszowe, błędne udziały, walidacja dat i godzin oraz 200 losowych scenariuszy rozliczeń. Backend: hashowanie, sesje, role, zaproszenia, idempotencja, konflikty, tombstone, głosowanie, wydatki, budżety, spłaty, historię, kopie bazy, proxy, WSGI i trwałość danych.

Android: CRUD wydatku i zmiana salda, błędy formularza, zachowanie formularza po odtworzeniu aktywności, ponowne głosowanie, odtworzenie danych i główne widoki, limity budżetu, formularz spłaty, miejsce oraz historię, wysokość mapy i zachowanie cache pogody po błędzie sieci. Integracja REST: dwa oddzielne konta/repozytoria, lokalna kolejka, równoczesna edycja, wybór obu sposobów rozwiązania konfliktu, usunięcia i cofnięcie dostępu, budżet, miejsce, spłatę i autorów historii.

Środowisko i dowody

Emulator Medium_Phone, Android 10 / API 29, ARM64, 1080 × 2400. Kompilacja: SDK 36.1, JDK 21 i Gradle 9.3.1. Testy backendu wykonano na Pythonie 3.13. Raporty XML, raport Lint i logi znajdują się w output/reports.

Ograniczenia weryfikacji

Nie wykonano testów na fizycznym telefonie, audytu bezpieczeństwa ani testów obciążeniowych. Interwały powiadomień i pracy w tle podlegają ograniczeniom Androida. Końcowy Lint uruchomiono offline: 1 ostrzeżenie wersji Lifecycle. JavaScript jest wymagany przez lokalny Leaflet; WebView nie ma mostka do Androida. Sprawdzenia dostępności nowszych wersji przez sieć nie należą do tego przebiegu.

Bezpieczeństwo i zakres odpowiedzialności

Sesja AES-GCM/Keystore; prywatna baza lokalna bez SQLCipher; wyłączone backup i transfer danych. Serwer: PBKDF2-SHA256, losowe tokeny ważne 7 dni, kontrola dostępu i limiter logowania. Backend jest dydaktyczny i wymaga własnego hostingu HTTPS do pracy poza lokalnym środowiskiem.

Organizacja i oddanie

Podział folderów

Folder	Odpowiedzialność
backend/	API, konta, uprawnienia, synchronizacja i testy
app/.../domain/	Modele, walidacja i rozliczenia
app/.../data/	Room, API, sesja, ustawienia i kolejka
app/.../ui/screens/	Oddzielne foldery funkcji: auth, trips, trip, plan, tasks, ideas, money, editor, members, conflicts, settings, budget, activity, places
app/src/test/ oraz androidTest/	Testy Kotlin, Compose i rzeczywistego REST
docs/ oraz scripts/	Dokumentacja, Mermaid, screeny i odtwarzalne budowanie
output/	APK, PDF, raporty oraz paczka projektu

Proponowany podział odpowiedzialności

Osoba 1: backend, konta, historia i hosting. **Osoba 2:** interfejs, plan, zadania, mapa i pogoda. **Osoba 3:** koszty, kategorie, budżety, spłaty oraz synchronizacja. Każdy obszar obejmuje testy i dokumentację. Podział umożliwia niezależne rozwijanie modułów i wspólną integrację.

Repozytorium Git

Projekt ma lokalne repozytorium i rzeczywistą historię etapów pracy. Nie wskazano zdalnego repozytorium, dlatego dokumentacja nie zawiera fikcyjnego publicznego linku. Po publikacji należy dopisać adres repozytorium. Dołączony Git bundle zachowuje historię.

Materiały do przekazania

Źródła Android i backendu, Gradle Wrapper, APK debug, dokumentacja PDF/Markdown, diagramy, schemat Room, screeny i raporty testów. Paczka pomija lokalne ścieżki SDK, cache, tokeny i bazy testowe.

Rozbudowa

Możliwe kolejne kroki: większe testy urządzeniowe, paginacja synchronizacji, testy hostingu publicznego, odzyskiwanie hasła i weryfikacja adresu e-mail. Funkcje te nie są przedstawiane jako już zaimplementowane.